```
$token = strtok($text, $delimiters);

while ($token !== false) {
    print "token: {$token}" . PHP_EOL;
    $token = strtok($delimiters);
}

Output:
token: Hi
token: there,
token: and
token: welcome
token: to
token: tokens!
```

`strtok()` requires initialization with two arguments when first called; the first argument is the string to be tokenized, and the second defines the characters (remember: byte values) to be treated as delimiters. `strtok()` simply walks along the string until it hits a delimiter character (in our case above that's a space or a newline); it returns the characters it has found so far and waits for further instructions. As you can see above, we're required to make more calls to `strtok()` to obtain subsequent tokens (each time passing in the delimiters again), and `strtok()` will eventually return `false` when there are no more tokens found in the string. To summarize, call `strtok()` with two arguments to initialize (the string being analyzed is held in memory between calls) and call `strtok()` with a single argument to process the in-memory string. If at any point `strtok()` receives two arguments, this will reset the in-memory string, and the process starts again. The analysis is complete when `strtok()` returns `false`.

The ability to switch delimiters between calls allows for some flexibility in parsing; here is an example:

```
$query = 'foo=bar&cat=miaow&dog=bark';
$delims = ['=', '&'];
$index = 0;

$token = strtok($query, $delims[$index]);
```